

**MARS ROVER MANIPAL
RESEARCH AI TASKPHASE
Report 6**

Adarsh Inaganti
240905294
adarshinaganti006@gmail.com

1. Review questions

1.1 What are the types of hypotheses?

1. Null and alternative hypotheses

- ***Null hypothesis***

- This hypothesis states that there is no effect, no significant difference, or no relationship between the variables being studied.
- It is the default assumption that researchers try to disprove.

- ***Alternative hypothesis***

- This hypothesis states the opposite of the null. It proposes that there is an effect, a significant difference, or a relationship.
- This is typically the researcher's actual prediction.

2. Directional and non-directional hypotheses

- ***Directional hypothesis (one-tailed)***

- This predicts not only that there is a relationship but also specifies the direction of that relationship or difference.

- ***Non-directional hypothesis (two-tailed)***

- This predicts that there is a relationship or difference, but it does not specify the direction.
- It just states that the groups will be different.

3. Simple and complex hypotheses

- ***Simple hypothesis***

- This predicts a relationship between a single independent variable (the cause) and a single dependent variable (the effect).

- ***Complex hypothesis***

- This predicts a relationship between two or more independent variables, dependent variables, or both.

4. Associative and casual hypotheses

- ***Associative hypothesis***

- This suggests that two variables are related or correlated, but it does not imply a cause-and-effect relationship.
- A change in one variable is associated with a change in the other.

- **Casual hypothesis**
 - This proposes a direct cause-and-effect relationship between variables.
 - It suggests that a change in the independent variable will cause a change in the dependent variable.

1.2 What are the advantages and disadvantages of using the sigmoid function?

1. Advantages

- **Probabilistic interpretation:** The most significant advantage is its output range. The sigmoid function "squashes" any real-valued input into a range between 0 and 1. This is extremely useful for models where the output needs to be interpreted as a probability.
- **Use case:** It's the standard activation function for the output layer in binary classification problems. An output of 0.85 can be interpreted as an 85% probability that the input belongs to the positive class.
- **Smooth and differentiable:** The function is smooth and its derivative (or gradient) is easy to calculate. This is a crucial property for training neural networks using backpropagation, which relies on gradients to update model weights.
- **Monotonic function:** The function is monotonic, meaning it's either entirely non-increasing or non-decreasing. Its derivative is always positive, which can help in gradient-based optimization.

2. Disadvantages

- **Vanishing gradient problem:**
 - The graph of the sigmoid function becomes very flat at both ends (for very large positive or negative inputs).
 - In these flat regions, the derivative (gradient) is extremely close to zero.
 - During backpropagation, gradients are multiplied across layers.
 - If many neurons are in these flat regions, their tiny gradients are multiplied together, causing the overall gradient for the early layers of the network to vanish.
 - The weights of the early layers don't get updated, and the network stops learning effectively.
- **Not zero-centered**
 - The output of the sigmoid function is not centered around 0 (centered at 0.5, with a range of [0, 1]).

- This can cause issues during weight updates. If the inputs to a neuron are always positive, the gradient updates for the weights will all be in the same direction.
- This can lead to inefficient, zig-zagging convergence during the training process. Functions like the hyperbolic tangent (tanh), which outputs in a $[-1, 1]$ range, solve this problem.
- **Computationally expensive**
 - Compared to its modern alternative, the Rectified Linear Unit (ReLU), the sigmoid function is more computationally expensive due to the exponential operation e^{-x} .
 - ReLU simply involves a $\max(0, x)$ operation, which is much faster.

1.3 How can we use a sigmoid function to predict multiple classes instead of only two?

1. For mutually exclusive classes (multi-class)

- Instead of the sigmoid function, the softmax function is used, which is a generalization of the sigmoid function.
- The softmax function takes the entire vector of outputs and squashes them into a probability distribution where all the outputs are between 0 and 1, and their sum equals 1.

2. For non-mutually exclusive classes (multi-label)

- The sigmoid function is used directly.
- A separate sigmoid function is applied independently to each output neuron.
- Each sigmoid output is a probability (between 0 and 1) for that one class, completely independent of the others. The probabilities will not sum to 1.

1.4 What does it mean for an activation function to be zero-centered?

An activation function is zero-centered if its range of output values is roughly centered around zero.

The zig-zag problem

- Consider an activation function that is not zero-centered, like ReLU or Sigmoid. All of its outputs are always positive. These positive activations are the inputs to the next layer.
- When the gradients are calculated for the weights of that next layer during backpropagation, the gradient will be proportional to this activation.

- Since the outputs are always positive, the gradient will have the same sign for all weights in that layer (either all positive or all negative). This forces the weights to update in a constrained way.

A zero-centered function, like tanh, solves this problem.

- Because the outputs (activations) can be positive or negative, the gradients for the weights in the next layer are not constrained.
- The updates can be positive for some weights and negative for others, allowing the model to find the optimal path much more efficiently and converge faster.

1.5 What is regularization and why is it used? When is it used? What are the types of regularization?

1. What is regularization?

- Regularization is a set of techniques used in machine learning to prevent overfitting.
- It is like adding a penalty for model complexity.
- A model that is too complex (overfit) learns the noise and random fluctuations in the training data, not just the underlying patterns.
- This causes it to perform poorly on new, unseen data.
- Regularization discourages this complexity, forcing the model to be simpler and more general which helps it perform better on data it hasn't seen before.

2. Why is it used?

- The main goal is to solve the problem of overfitting.
- It introduces a small amount of bias (simplification) to significantly reduce this variance.
- This trade-off almost always results in a model that generalizes better to the real world.

3. When is it used?

- When the model performs great on training data but poorly on test/validation data (overfitting).
- When there are a large number of features. With many features, it's easy for the model to find complex, coincidental patterns that are just noise. Regularization can help reduce the influence of less important features.
- When training deep neural networks. Large networks with many layers are extremely flexible and have a very high risk of overfitting, so regularization (like Dropout) is a standard part of their training.

4. Types of regularization

- **L1 regularization (lasso regression)**
 - It adds a penalty to the loss function equal to the sum of the absolute values of the model's weights (coefficients).
 - This penalty forces weaker, less important features to have their weights go exactly to zero.
 - Best for feature selection because it zeros out irrelevant features. It automatically creates a simpler, more interpretable model that only uses the most important inputs.
- **L2 regularization (ridge regression)**
 - It adds a penalty equal to the sum of the squared values of the weights.
 - This penalty forces weights to be small, but it does not force them to be zero. It just shrinks them all toward zero. Less extreme than L1.
 - Best for overall performance and stability as it is the most common type of regularization and generally improves model generalization without eliminating features entirely.

1.6 What is the difference between normalization and standardization?

1. Normalization (min-max scaling)

- It scales all data points into a fixed, bounded range.
- **Formula:**

$$X' = \frac{X - X_{min}}{X_{max} - X_{min}}$$

- All values will be between 0 and 1. The minimum value in the original data becomes 0, and the maximum value becomes 1.
- Very sensitive to outliers.
- A single extreme outlier will cause all other data points to be squashed into a tiny part of the [0, 1] range, which can hurt model performance.

2. Standardization (Z-score scaling)

- It centers the data around a mean of zero and scales it based on its standard deviation.
 - **Formula:**
- $$X' = \frac{X - \mu}{\sigma}$$
- The resulting range is unbounded.
 - Very low sensitivity to outliers.

- Outliers are still present, but they don't dominate the scaling of other features in the same way as in normalization. The mean and standard deviation are less sensitive to outliers than the min and max values.

3. When to use which?

- **Normalization**
 - The data does not follow a Gaussian (bell curve) distribution.
 - The algorithm being used does not assume any distribution, such as:
 - K-Nearest Neighbors (KNN)
 - K-Means Clustering
 - While working with neural networks, activation functions (like sigmoid) expect inputs between 0 and 1.
 - While working with image processing, where pixel values are already bounded (0 to 255).
- **Standardization**
 - The data follows a Gaussian (bell curve) distribution.
 - The algorithm being used assumes your data is centered at zero, such as:
 - Linear Regression
 - Logistic Regression
 - Linear Discriminant Analysis (LDA)
 - Principal Component Analysis (PCA)
 - The dataset has outliers. Standardization is safer and more robust to them than normalization.

1.7 Is accuracy a good evaluation metric for classification tasks?

- No, accuracy is often not a good metric for classification tasks, and it can be highly misleading.
- The only time accuracy is a reliable metric is when your dataset is perfectly balanced (e.g., 50% of samples are class A and 50% are class B) and the cost of all types of errors is the same.
- Accuracy's biggest flaw is that it fails completely with imbalanced datasets.
- Accuracy gives us nothing about how the model is right or wrong. It treats all errors as equal. In reality, some errors are much worse than others:
 - **False positive (type I error):** Predicting "disease" when the patient is "healthy." This causes unnecessary panic and more tests.
 - **False negative (type II error):** Predicting "healthy" when the patient has the "disease." This is a critical, potentially life-threatening failure.
- Accuracy hides these details. For this, we use a confusion matrix to see the four possible outcomes.

Better metrics

1. Precision

$$Precision = \frac{True\ positives}{True\ positives + False\ positives}$$

Used when the cost of false positives is high.

2. Recall

$$Recall = \frac{True\ positives}{True\ positives + False\ negatives}$$

Used when the cost of false negatives is high.

3. F1 score

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

Used when a balance between precision and recall is needed.

1.8 Is loss used as an evaluation metric?

- Loss is not reported as a final evaluation metric.
- Loss is used to tell us how confident or how wrong the model is. It is used for training the model.
- It is not used to evaluate the performance of a model. It is instead used to diagnose overfitting by comparing training and validation loss.

1.9 What are the types of encoding and which is used where?

- Encoding is the process of converting data from one format to another.
- In machine learning, this almost always means converting categorical features into a numerical format so that algorithms can understand and process them.
- **Nominal data**
 - **One-hot encoding**
 - Creates a new binary (0 or 1) column for each unique category.
 - This is the most common and safest method, especially for linear models (like Logistic Regression) and neural networks. It prevents the model from assuming a false relationship.
 - It can create a huge number of new features if there are many unique categories (high cardinality), which can slow down training.

- **Binary encoding**
 - It first converts categories to integers, then converts those integers into binary code, and finally splits the binary digits into separate columns.
 - A great choice for features with high cardinality (e.g., 50+ unique categories), as it creates far fewer columns than one-hot encoding ($\log_2(n)$ columns vs. n columns).
- **Hash encoding**
 - Uses a hashing function to convert a category name into a numerical index of a fixed-size vector.
 - Used for extremely high cardinality features (e.g., millions of unique values, like user IDs or zip codes) or in online learning systems where an encoder cannot be pre-fit.
 - Collisions (where two different categories get mapped to the same index) are possible, which can add noise. It is also not interpretable as we can't easily tell what the resulting features mean.
- **Ordinal data**
 - **Label encoding**
 - Converts each unique category into a unique integer.
 - This should only be used for ordinal data. The numerical relationship (e.g., $3 > 2$) must match the real-world relationship of the categories.
- **Using the target variable**
 - **Target encoding**
 - Replaces each category with the average value of the target variable for that category.
 - Great for high-cardinality features because it captures information about the target variable in a single column.
 - Prone to overfitting.

1.10 What is the use of seed?

- A seed in Machine Learning (ML) is used to initialize a pseudorandom number generator (PRNG).
- The primary reason for using a seed is to ensure reproducibility.
- By setting a specific seed at the beginning of the script, there is a guarantee that the PRNG will produce the exact same sequence of random numbers every time the code is run.

- This means:
 - The data splits will be identical.
 - The initial model weights will be the same.
 - Any other random process will follow the same pattern.

1.11 What is R^2 score and adjusted R^2 score?

Both the R-squared (R^2) and Adjusted R-squared are metrics that measure how well a regression model fits the data, but the Adjusted R^2 provides a more accurate assessment, especially when the model has multiple independent variables.

1. R^2 score

- The R-squared score, also known as the coefficient of determination, measures the proportion of the variance in the dependent variable that is predictable from the independent variable(s).
- It tells us what percentage of the variation in the output variable can be explained by the input variable(s).
 - **Range:** 0 to 1 (or 0% to 100%).
 - **Interpretation:** An R^2 of 0.82 means that 82% of the variation in the dependent variable can be explained by the model.
- The major flaw of R^2 is that it almost always increases as you add more independent variables to the model, even if those variables are not useful for predicting the outcome.

2. Adjusted R^2 score

- The Adjusted R-squared is a modified version of R^2 that has been adjusted for the number of predictors in the model.
- Unlike R^2 , the Adjusted R^2 will only increase if the new variable improves the model more than would be expected by chance.
- This makes it a more reliable metric when you are comparing models with different numbers of independent variables.
 - **Penalty for complexity:** The Adjusted R^2 penalizes the model for having extra variables that are not contributing to the model's explanatory power.
 - **Can decrease:** If you add a useless variable to the model, the Adjusted R^2 can actually decrease.

1.12 What are all the possible metrics of linear and logistic regression?

1. Linear regression metrics

- **Mean squared error (MSE)**

- The average of the squared differences between the predicted values and the actual values.

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

- It heavily penalizes large errors.
- Units are squared.

- **Root mean squared error (RMSE)**

- The square root of MSE.

$$RMSE = \sqrt{MSE}$$

- Has the same units as the target variable, making it more interpretable than MSE.
- Still sensitive to large errors (outliers).

- **Mean absolute error (MAE)**

- The average of the absolute differences between predicted and actual values.

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

- Less sensitive to outliers compared to MSE/RMSE.
- Represents the average magnitude of the errors.

- **R-squared (R^2)**

- The proportion of the variance in the dependent variable that is predictable from the independent variables.
- Ranges from 0 to 1 (or 0% to 100%).
- Higher values indicate a better fit (e.g., $R^2=0.8$ means 80% of the variance is explained).
- Can be misleading as it always increases when more variables are added.

- **Adjusted R^2**

- A modified version of R^2 that adjusts for the number of predictors in the model.
- Penalizes the score for adding irrelevant predictors.
- Better for comparing models with different numbers of features.

2. Logistic regression metrics

○ **Accuracy**

- The proportion of total predictions that were correct.

$$\text{Accuracy} = \frac{\text{True positives} + \text{True negatives}}{\text{Total predictions}}$$

- Simple but can be misleading, especially with imbalanced datasets.

○ **Confusion matrix**

- A table summarizing classification performance:

- **True positives (TP):** Correctly predicted positive.
- **True negatives (TN):** Correctly predicted negative.
- **False positives (FP):** Incorrectly predicted positive (Type I error).
- **False negatives (FN):** Incorrectly predicted negative (Type II error).

○ **Precision**

- Out of all instances predicted as positive, how many actually were positive?

$$\text{Precision} = \frac{\text{True positives}}{\text{True positives} + \text{False positives}}$$

- Important when the cost of false positives is high.

○ **Recall**

- Out of all actual positive instances, how many were correctly predicted?

$$\text{Recall} = \frac{\text{True positives}}{\text{True positives} + \text{False negatives}}$$

- Important when the cost of false negatives is high.

○ **F1-score**

- The harmonic mean of Precision and Recall.

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

- Useful when you need a balance between Precision and Recall, especially with imbalanced classes.

○ **Specificity**

- Out of all actual negative instances, how many were correctly predicted?

$$\text{Specificity} = \frac{\text{True negatives}}{\text{True negatives} + \text{False positives}}$$

○ **ROC (receiver operating characteristic) curve**

- A plot showing the True Positive Rate (Recall) vs. the False Positive Rate (1 - Specificity) at various classification thresholds.

- **AUC (area under ROC curve)**
 - A single number summarizing the ROC curve.
 - Ranges from 0 to 1.
 - Higher is better (1 = perfect classifier, 0.5 = random guessing).
 - Measures the model's ability to distinguish between positive and negative classes across all thresholds.
- **Log loss (logistic loss or cross-entropy loss)**
 - Measures the performance of a model whose output is a probability value between 0 and 1.
 - It penalizes heavily for predictions that are confident and wrong.
 - Lower values are better.
 - This is often the loss function optimized during training.

1.13 What is ROC and AUC?

1. ROC (receiver operating characteristics) curve

- The ROC curve is a graph that illustrates the diagnostic ability of a binary classifier system as its discrimination threshold is varied.
 - It plots two parameters:
 - **True positive rate (TPR) or recall**
 - It measures the proportion of actual positive cases that were correctly identified. Plotted on the y-axis.
- $$TPR = \frac{\text{True positives}}{\text{True positives} + \text{False negatives}}$$
- Plotted on the y-axis.
 - **False positive rate (FPR)**
 - It measures the proportion of actual negative cases that were incorrectly identified as positive.
- $$FPR = \frac{\text{False positives}}{\text{False positives} + \text{True negatives}}$$
- Plotted on the x-axis.
 - A curve that bows towards the top-left corner indicates a better-performing model. This means achieving a high TPR (catching positives) while keeping the FPR (false alarms) low.
 - The diagonal line (y=x) represents a model with no discriminative ability, equivalent to random guessing.

2. AUC (area under ROC curve)

- The AUC is simply the area under the ROC curve.
- Ranges from 0 to 1.

- **Interpretation:**
 - **AUC = 1:** Perfect classifier. It achieves 100% TPR with 0% FPR.
 - **AUC = 0.5:** The model has no discriminative ability (same as random guessing). This corresponds to the diagonal line on the ROC plot.
 - **AUC < 0.5:** The model performs worse than random guessing (it might be predicting classes inversely).
 - **Higher AUC (closer to 1):** Indicates a better model at distinguishing between the positive and negative classes across all thresholds.
- AUC provides a single scalar value summarizing the model's performance across all possible classification thresholds, making it useful for comparing different models.

1.14 What is the ideal test-train split ratio?

- There is no single ideal ratio, but the most common splits are:
 - 80% for training and 20% for testing (80/20)
 - 70% for training and 30% for testing (70/30)
- **Factors influencing the split ratio**
 - **Total dataset size**
 - **Large datasets (e.g., millions of samples):** A smaller percentage might be used for testing (like 90/10 or even 99/1) because even 1% can represent a large number of samples, providing a reliable evaluation.
 - **Small datasets:** A larger percentage is needed for testing (like 70/30 or 60/40) to ensure the test set is representative enough to evaluate the model's generalization properly. The training data should also be maximized.
 - **Complexity of the problem/model:** More complex models or problems might benefit from having more training data (e.g., leaning towards 80/20 or 90/10 if the dataset is large enough).
 - **Use of a validation set:** Often, a three-way split is used: Train/Validation/Test. Common ratios include 60/20/20 or 70/15/15. The validation set is crucial for tuning hyperparameters and preventing overfitting before the final evaluation on the test set.

1.15 What is softmax? What is it used for?

- Softmax is a mathematical function that converts a vector of raw numerical scores (often called "logits") into a probability distribution.
- This means it takes a list of numbers and transforms them so that:
 - Each number is between 0 and 1.
 - All the numbers in the list sum up to 1.
- Softmax is primarily used as the activation function in the output layer of a neural network for multi-class classification problems.
- If there is a model that needs to decide which one category an input belongs to out of multiple possible categories, softmax is used to:
 - Take the model's raw output scores for each class.
 - Convert these scores into probabilities.
- The class with the highest probability is then chosen as the model's prediction.
- This makes the model's output easily interpretable as the likelihood or confidence assigned to each possible class.

1.16 What is the difference between loss and cost?

- **Loss function:** Calculates the error for a single training example. It measures how far the model's prediction is from the actual target value for that one instance.
- **Cost function:** Calculates the average loss over the entire training dataset (or sometimes a batch of data). It gives an aggregate measure of how well the model is performing across all the examples it has seen.
- The cost function is the average of the loss function values.
- The goal during training is typically to minimize the cost function, which implicitly minimizes the loss for individual examples on average.

1.17 What is a metric?

- A metric in ML is a quantitative measure used to evaluate the performance of the model.
- Metrics are used to:
 - **Assess performance:** Understand how good or bad the model is.
 - **Compare models:** Decide which model architecture or set of hyperparameters performs better.
 - **Monitor progress:** Track improvement during training (often using validation metrics).
- Metrics are primarily for human interpretation and model assessment.

- **Examples:**
 - **Classification:** Accuracy, precision, recall, F1-Score, AUC
 - **Regression:** RMSE (Root Mean Squared Error), MAE (Mean Absolute Error), R^2

1.18 What is Huber loss?

- Huber loss is a loss function used in regression tasks that is less sensitive to outliers than Mean Squared Error (MSE).
- It is still differentiable at 0, unlike Mean Absolute Error (MAE).
- It essentially combines the best properties of both MSE and MAE.
- **How it works**
 - Huber loss behaves like MSE for small errors and like MAE for large errors.
 - The transition point between these two behaviors is controlled by a parameter called delta.
 - **Formula:**

$$L_{\delta}(y, f(x)) = \begin{cases} \frac{1}{2}(y - f(x))^2 & \text{for } |y - f(x)| \leq \delta \\ \delta|y - f(x)| - \frac{1}{2}\delta^2 & \text{for } |y - f(x)| > \delta \end{cases}$$

- y : actual value
- $f(x)$: predicted value
- δ : threshold parameter

- **Why should Huber loss be used?**
 - **Robustness to outliers:** Like MAE, it doesn't overly penalize large errors caused by outliers, leading to more robust models when outliers are present.
 - **Differentiability:** Unlike MAE, which has a sharp corner at zero, Huber loss is smooth and differentiable everywhere (specifically, it has a derivative of 0 at 0), which is beneficial for optimization algorithms like gradient descent.
 - **Balance:** It provides a good balance between the sensitivity of MSE to large errors and the non-differentiability of MAE.

1.19 When do we use normalization?

- **When the algorithm does not assume any specific distribution of the data.** This is common for algorithms that rely on distances between data points, such as:
 - K-Nearest Neighbors (KNN)
 - K-Means Clustering
- **When the algorithm requires inputs within a bounded range.** This is often the case for:
 - **Neural networks:** Activation functions like sigmoid and tanh work best with inputs roughly between 0 and 1 or -1 and 1. Normalization helps keep the inputs in this range, aiding convergence.
 - **Image processing:** Pixel values are often scaled to [0, 1] before being fed into models.
- **When there are features on vastly different scales but know there are no significant outliers.** Normalization brings all features to the same scale, preventing features with larger values from dominating distance calculations or gradient updates.

1.20 What are TP, FP, TN, and TP?

1. TP (True Positive)

- **Actual:** Positive
- **Predicted:** Positive
- The model correctly predicted a positive case. (e.g., correctly identified a spam email as spam).

2. FP (False Positive)

- **Actual:** Negative
- **Predicted:** Positive
- The model incorrectly predicted a negative case as positive (a "false alarm"). (e.g., predicted a normal email as spam).
- Also known as a Type I error

3. TN (True Negative)

- **Actual:** Negative
- **Predicted:** Negative
- The model correctly predicted a negative case. (e.g., correctly identified a normal email as not spam).

4. FN (False Negative)

- **Actual:** Positive
- **Predicted:** Negative
- The model incorrectly predicted a positive case as negative (a "miss"). (e.g., predicted a spam email as not spam).
- Also known as a Type II error.

1.21 What is class imbalance and how do you handle it?

- Class imbalance occurs in classification problems when the number of examples for different classes is significantly unequal.
- One class (the majority class) has many more instances than the other(s) (the minority class).
- Standard machine learning models often assume balanced class distributions.
- When faced with imbalance, they tend to become biased towards the majority class.
- They can achieve high accuracy simply by always predicting the majority class, while completely failing to identify the minority class (which is often the class we are the most interested in, like fraud or a rare disease).
- This makes accuracy a poor metric for imbalanced datasets.
- **How to handle class imbalance**
 - **Resampling techniques:** Modify the dataset to make it more balanced.
 - **Undersampling:** Randomly remove instances from the majority class.
 - **Pros:** Can speed up training.
 - **Cons:** Might discard useful information from the majority class.
 - **Oversampling:** Randomly duplicate instances from the minority class.
 - **Pros:** Doesn't lose information.
 - **Cons:** Can lead to overfitting on the duplicated minority samples.
 - **Synthetic minority over-sampling technique (SMOTE)**
 - A more advanced oversampling method. Instead of duplicating, it creates new synthetic minority class examples by interpolating between existing minority instances.
 - This helps avoid overfitting.

- **Algorithmic Approaches (Cost-Sensitive Learning):** Modify the learning algorithm to give more importance to the minority class.
 - **Class weights**
 - Assign higher weights to the minority class during model training.
 - This means misclassifying a minority instance incurs a higher penalty, forcing the model to pay more attention to it.
 - Most modern ML libraries allow you to set class weights.
- **Use appropriate evaluation metrics:** Focus on metrics that give a better picture of performance on the minority class.
 - **Precision:** Good when the cost of false positives is high.
 - **Recall (Sensitivity):** Crucial when the cost of false negatives is high (often the case for minority classes).
 - **F1-Score:** A balance between precision and recall.
 - **AUC (Area Under the ROC Curve):** Measures the model's ability to distinguish classes across different thresholds.
 - **Precision-Recall Curve (and AUC-PR):** Often more informative than ROC-AUC for highly imbalanced datasets.
- **Ensemble methods:** Combine multiple models, often trained on different balanced subsets of the data.
 - **Balanced Random Forest:** Modifies the Random Forest algorithm to balance the data within each bootstrap sample.
 - **EasyEnsemble / BalanceCascade:** Train multiple classifiers on different undersampled majority class subsets and combine their predictions.
- **Anomaly detection:** Reframe the problem. Instead of classifying, treat the minority class as "anomalies" and use anomaly detection algorithms to find them.

1.22 What is a precision-recall curve?

- A Precision-Recall (PR) curve is a graph that evaluates the performance of a binary classification model, especially when dealing with imbalanced datasets.
- It plots two metrics:
 - Precision on the y-axis.
 - Recall on the x-axis.

- **Interpretation**

- **Ideal point:** The top-right corner (Recall=1, Precision=1) represents a perfect model.
- **Curve shape:** A curve that stays closer to the top-right corner indicates better performance. It means the model can achieve high precision (few false positives) while maintaining high recall (few false negatives).
- **Baseline:** For a highly imbalanced dataset with P positive examples and N negative examples, a random classifier would achieve a Precision equal to the fraction of positives $P / (P + N)$. This value often serves as a baseline on the PR curve plot.
- **Area under the curve (AUC-PR or PR-AUC):** The area under the PR curve (often calculated using the Average Precision score) provides a single number summarizing the model's performance across all thresholds. A higher AUC-PR (closer to 1) is better.

1.23 What is bias and variance?

1. Bias

- Bias refers to the error introduced by approximating a real-world problem, which may be complex, with a too-simple model.
- **High bias**
 - Occurs when the model makes strong assumptions about the data (e.g., assuming a linear relationship when it's actually curved).
 - This leads to underfitting, where the model fails to capture the underlying patterns in the data, performing poorly on both the training and test sets.
- This results in systematic errors, consistently missing the true relationship.

2. Variance

- Variance refers to the error introduced because the model is too sensitive to small fluctuations in the training data.
- **High variance**
 - Occurs when the model is overly complex and learns not just the underlying patterns but also the noise specific to the training set.
 - This leads to overfitting, where the model performs extremely well on the training data but poorly on new, unseen data (test set).

- This results in the model changing drastically with different training datasets, leading to inconsistent predictions.

3. The bias-variance tradeoff

- There is typically a trade-off between bias and variance:
 - **Simple models:** Tend to have high bias and low variance.
 - **Complex models:** Tend to have low bias and high variance.

2. Support vector machines

2.1 Introduction

2.1.1 Definition

- A support vector machine (SVM) is a popular and powerful supervised machine learning algorithm used primarily for classification problems. It's also capable of handling regression tasks.
- The main goal of an SVM is to find the best possible line or boundary that separates data points belonging to different classes in a dataset.

2.1.2 Core concepts

- 1. Hyperplane:** It is the line or boundary which is found by the SVM model.
 - In a 2D space, the hyperplane is a line.
 - In a 3D space, it is a flat plane.
 - In spaces with more than 3 dimensions, it's called a hyperplane.
- 2. Margin**
 - An SVM doesn't just draw any line, it finds the line that creates the widest possible "street" between the two classes.
 - This "street" is called the margin. The goal is to maximize the width of this margin, which makes the model more confident in its classifications.
- 3. Support vectors:** Data points (the dots) that are closest to the hyperplane.
 - They are the critical points that "support" the hyperplane and define the width of the margin.
 - If we were to move any of these support vectors, the position of the hyperplane would change.
 - The SVM ignores all other points, making it very memory-efficient.

2.2 Kernel function

2.2.1 Definition

A kernel function is a mathematical function that computes the similarity between two data points in a (possibly high-dimensional) feature space without explicitly transforming the data into that space.

The kernel trick is about replacing the dot product $(x_i \cdot x_j)$ with a kernel function $K(x_i, x_j)$. This function computes the dot product in a high-dimensional feature space without ever creating or storing the high-dimensional vectors.

2.2.2 Linear kernel

The linear kernel is the simplest kernel function used in SVMs.